Claude Code Implementation Brief

Project: AzureDocumentAI-DotNet | **Target:** small working MVP in 20-30 minutes (when Azure resources already exist).

Instruction to Claude Code: Implement this project end-to-end in the current folder. Prioritize a working, clean demo over extra features. Do not ask unnecessary questions. Do not pretend Azure calls have succeeded without valid credentials. Run a build and provide exact run/test commands.

1. Goal and reference

Recreate the essential flow in the supplied diagram using **C# / ASP.NET Core Minimal API** instead of Python/FastAPI: PDF upload -> Azure AI Document Intelligence -> text, tables and key-value extraction -> Azure OpenAI chat completion -> normalized JSON response. No database is needed for the MVP; save JSON locally only if easy.

Reference: https://github.com/Sandesh-hase/SupplyChain-Document-Intelligence (use it for functional inspiration, not copy-pasted Python). The reference includes app.py and doc_processing.py. This is a fresh .NET implementation.

2. Exact scope - required

• One .NET 8 or later ASP.NET Core Web API project with Swagger/OpenAPI, if readily available.

• One endpoint: POST /api/documents/analyze accepting multipart/form-data with one PDF file (max 10 MB).

• Use Azure.AI.DocumentIntelligence with the prebuilt-layout model. Extract plain text, tables (rows/cells) and key-value pairs if present. Never fail merely because some fields are absent.

• Pass a concise, bounded extraction summary to Azure OpenAI via the official Azure/OpenAI .NET SDK or a documented Azure REST chat-completions endpoint. Ask for structured JSON with documentType, summary, parties, dates, totals and keyFacts (null/empty when unknown). Do not hallucinate missing values.

• Return one JSON payload containing filename, documentIntelligence result (text, tables, keyValuePairs) and aiNormalized result. Validate/parse the LLM JSON and handle malformed responses gracefully.

• Use environment variables for credentials. Add .env.example or README variables, but NEVER commit secrets, real .env, or uploaded private documents.

• Provide simple input validation, useful failure messages, cancellation token support if straightforward, and no unnecessary abstractions.

3. Keep the architecture tiny

```
AzureDocumentAI-DotNet/
  Program.cs
  Services/DocumentAnalysisService.cs
  Services/AzureOpenAiService.cs
  Models/DocumentAnalysisResponse.cs
  README.md
  .gitignore
  .env.example
```

Smallest maintainable structure wins; collapse files if that simplifies implementation. No Blazor, MongoDB, SQL, Docker, authentication, queues, vector search, MCP, tests framework, or deployment pipeline for this demo.

4. Azure configuration

```
DOCUMENT_INTELLIGENCE_ENDPOINT=...
DOCUMENT_INTELLIGENCE_KEY=...
AZURE_OPENAI_ENDPOINT=...
AZURE_OPENAI_API_KEY=...
AZURE_OPENAI_DEPLOYMENT=...
```

Use the deployed Azure OpenAI model name, not an assumed model ID. Follow currently installed SDK APIs and package signatures; do not invent methods. Use sensible timeouts and avoid printing secrets or full document contents in logs. Required resources: Azure AI Document Intelligence and a deployed Azure OpenAI chat-capable model.

5. Minimal demonstration

Provide an example invoice PDF (synthetic/non-sensitive if you can create one quickly) under samples/, or explain where to obtain a sample. Run the app locally, upload via Swagger or curl, and show JSON containing extracted text/tables and the AI-normalized summary. A small invoice with line items makes the table demo clear.

```
dotnet restore
dotnet build
dotnet run
# Use Swagger if available, otherwise curl -F "file=@samples/invoice.pdf"
```

If Azure credentials are unavailable, still finish a compiling solution, explain which checks were and were not run, and give the exact commands to test with real Azure resources. Do not fabricate successful integration tests.

6. README and GitHub readiness

Write a concise README with purpose, a 4-stage architecture diagram (Mermaid is fine), tech stack, prerequisites, Azure resource setup, environment-variable configuration, run steps, example response, limitations and attribution to the reference project. Include a clear explanation of what Document Intelligence handles versus what the LLM handles.

Initialize git and prepare the repository. If gh CLI is authenticated and repository creation is explicitly available, create a public repo named **AzureDocumentAI-DotNet** and push; otherwise provide copy-paste commands for the user to create/push it. Never overwrite an existing repository, force push, or expose credentials.

7. Definition of done

A successful MVP: dotnet build passes; POST accepts an invoice PDF; Document Intelligence extracts structured document content; Azure OpenAI transforms it into grounded, normalized JSON; README allows another developer to run it; no secrets are committed. Keep the focus on the full Azure integration path, not production hardening.

Time-box: 5 min scaffold; 10 min Document Intelligence; 8 min Azure OpenAI; 5 min README/build/GitHub. Prefer a smaller working version if blocked.